Reinforcement Learning Portfolio Management

Project Progress Report

RL Portfolio Management Project

October 15, 2026

1 Introduction

This project investigates whether a reinforcement learning agent can improve portfolio allocation across U.S. equity sectors. The agent observes historical market signals and its current holdings, then adjusts sector weights every five trading days. Portfolio returns, trading costs, borrowing costs, and weight drift are calculated daily. The objective is to maximize portfolio growth after the modeled implementation costs.

The current implementation is a research pipeline consisting of market-data collection, feature engineering, a custom portfolio environment, PPO training, saved model checkpoints, and development-validation comparisons against simple benchmarks. Three main model stages have been implemented: a long-only allocator, an unscaled long-short allocator, and a long-short allocator with scaled changes to portfolio weights. Repeated-seed experiments and an action-scale search have also been completed.

The downloaded market history ends on September 30, 2026. All performance figures below are historical development-validation results used during model development as the ETF experiments reserve the final two years for a later evaluation.

2 Technology Used

The project uses Python and Jupyter notebooks, with PyTorch supporting the neural networks and Stable-Baselines3 providing the Proximal Policy Optimization (PPO) implementation. Gymnasium supplies the interface for the custom allocation environment. NumPy and pandas handle numerical operations, date alignment, feature construction, and saved results. yfinance retrieves Yahoo Finance data. Matplotlib supports the analysis plots.

The workflow is organized into six numbered notebooks: data download, feature engineering, agent training, benchmark analysis, action-scale search, and the preliminary stock-fund experiment. The active settings and action/reward functions are defined in `notebooks/experiment_config.py`. Saved experiment folders contain checkpoints, configuration snapshots, chronological split summaries, daily validation records, and aggregate metrics. Grid-search runs save their individual settings as JSON.

Component	Pinned version	Role
Python	3.11 (tested)	Runtime documented in the requirements file
NumPy / pandas	1.26.4 / 2.2.3	Arrays, feature panels, and performance analysis
yfinance	0.2.61	Adjusted prices and preliminary stock screening
Gymnasium	0.29.1	Observation, action, and episode interfaces
Stable-Baselines3	2.3.2	PPO training and deterministic evaluation
PyTorch	2.4.0	Neural-network training
Matplotlib	3.9.2	Return, drawdown, and allocation plots
JupyterLab / ipykernel	4.2.5 / 6.29.5	Interactive experimentation

Table 1: Dependencies recorded in `requirements.txt`.

3 Data

3.1 Market history and investment universe

The main dataset contains daily observations for the nine original, plus SPY and the VIX as context series. SPY and VIX provide information to the policy but are not tradable assets in the current environment.

ETF	Sector
XLB	Materials
XLE	Energy
XLF	Financials
XLI	Industrials
XLK	Information Technology
XLP	Consumer Staples
XLU	Utilities
XLV	Health Care
XLY	Consumer Discretionary

Table 2: The current nine-asset allocation universe.

The saved raw file, `data/raw/yahoo_daily_adjusted.csv.gz`, covers January 4, 1999 through September 30, 2026. It contains 76,760 rows across 11 symbols and 6,980 trading dates. The download uses split- and dividend-adjusted prices (`auto_adjust=True`), with open, high, low, close, volume, and corporate-action fields retained. The simulator uses adjusted close-to-close returns. The processed file, `data/processed/market_features.csv.gz`, retains the same row count and adds the engineered features.

3.2 Features and observations

Let $P_{i,t}$ denote the adjusted closing price of asset i on trading day t, and let $N = 9$ be the number of tradable sector ETFs. All windows below count trading days: 5 days approximate one week, 21 days one month, and 63 days one quarter. These horizons provide different views of recent market behavior. The explanations below describe the design rationale; the project has not yet established that these features or window lengths are optimal.

Features for each tradable ETF. Each ETF contributes the following eight features:

- **5-day return (`return_5d`).**

$$r_{i,t}^{(5)} = \frac{P_{i,t}}{P_{i,t-5}} - 1.$$

This measures the price change over approximately one trading week. It gives the policy a short-horizon momentum signal that matches its five-day decision interval and can reveal recent changes before they dominate longer windows.

- **21-day return (`return_21d`).**

$$r_{i,t}^{(21)} = \frac{P_{i,t}}{P_{i,t-21}} - 1.$$

This measures approximately one month of performance. It gives a broader view of sector momentum than the weekly return and supplies the common horizon used for sector rankings, market-relative returns, and return dispersion.

- **63-day return (`return_63d`).**

$$r_{i,t}^{(63)} = \frac{P_{i,t}}{P_{i,t-63}} - 1.$$

This measures approximately one quarter of performance. Comparing it with the shorter returns helps the policy distinguish a sustained trend from a recent reversal or acceleration. All three return features are simple cumulative returns stored as decimals, rather than annualized returns.

- **21-day annualized volatility (`volatility_21d`).** First define the daily log return $\ell_{i,t} = \log(P_{i,t}/P_{i,t-1})$. Then

$$\bar{\ell}_{i,t} = \frac{1}{21}\sum_{k=0}^{20} \ell_{i,t-k}, \qquad \sigma_{i,t}^{(21)} = \sqrt{252}\sqrt{\frac{1}{20}\sum_{k=0}^{20}(\ell_{i,t-k} - \bar{\ell}_{i,t})^2}.$$

This matches the notebook's sample standard deviation of daily log returns, multiplied by $\sqrt{252}$. It measures recent return variability so the policy can distinguish sectors with similar momentum but different risk. The one-month window keeps the estimate responsive to recent conditions; annualization puts it on a familiar yearly scale.

- **Drawdown relative to the 63-day high (`rolling_drawdown_63d`).**

$$D_{i,t}^{(63)} = \frac{P_{i,t}}{\max_{0\le k\le 62} P_{i,t-k}} - 1.$$

This is zero at the rolling high and negative below it. It describes how far a sector has fallen from its recent peak, adding information about its price path that an endpoint return alone does not capture. The quarterly window gives context beyond the most recent month. This feature is the current distance from a rolling high, rather than the worst drawdown within the window.

- **Cross-sectional rank of the 21-day return (`cross_sectional_rank_return_21d`).**

$$Q_{i,t} = \frac{\mathrm{rank}_{j=1,\dots,N}(r_{j,t}^{(21)})_i}{N}.$$

Ranks are ascending, with average ranks for ties, matching pandas' percentile-rank calculation. With no ties, the weakest sector receives 1/9 and the strongest receives 1. This identifies relative sector leadership even when all sectors rise or fall together. Ranking also makes the ordering less sensitive to unusually large return magnitudes.

- **21-day return relative to SPY (`market_relative_return_21d`).**

$$M_{i,t}^{(21)} = r_{i,t}^{(21)} - r_{\mathrm{SPY},t}^{(21)}.$$

This measures whether a sector has outperformed or underperformed the broad market over the month. For example, a sector can have a positive raw return while still lagging SPY. The feature helps separate sector leadership from a market-wide rally; it is a return difference, not a beta-adjusted alpha.

- **63-day rolling market beta (`rolling_market_beta_63d`).**

$$\beta_{i,t}^{(63)} = \frac{\mathrm{Cov}_{63}(r_i^{(1)}, r_{\mathrm{SPY}}^{(1)})_t}{\mathrm{Var}_{63}(r_{\mathrm{SPY}}^{(1)})_t}, \qquad r_{i,t}^{(1)} = \frac{P_{i,t}}{P_{i,t-1}} - 1.$$

Covariance and variance use the latest 63 daily simple returns, including day t, with sample estimates. Beta describes a sector's recent sensitivity to market movements. It supplies market-exposure information alongside raw returns and volatility. The quarterly window provides more observations for estimating this relationship than a one-month window.

SPY context features. SPY contributes its 5-, 21-, and 63-day returns, 21-day annualized volatility, and 63-day rolling drawdown, calculated with the same formulas above using SPY prices. These five inputs describe broad-market trend, variability, and distance from recent highs, helping the policy interpret sector signals in their market context. SPY is an observation source rather than a tradable asset in the current environment.

Market-wide features. The following five features summarize the sector universe and the volatility environment:

- **Sector breadth above the 63-day moving average (`sector_breadth_above_sma_63d`).**

$$\mathrm{SMA}_{i,t}^{(63)} = \frac{1}{63}\sum_{k=0}^{62} P_{i,t-k}, \qquad B_t = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\left\{P_{i,t} > \mathrm{SMA}_{i,t}^{(63)}\right\}.$$

This is the fraction of sectors above their quarterly moving averages. Values near one indicate widespread participation in an upward trend; values near zero indicate broad weakness. It helps distinguish a market move supported by many sectors from one driven by only a few.

- **Dispersion of 21-day sector returns (`sector_return_dispersion_21d`).**

$$\overline{r}_t^{(21)} = \frac{1}{N}\sum_{i=1}^{N} r_{i,t}^{(21)}, \qquad S_t = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(r_{i,t}^{(21)} - \overline{r}_t^{(21)})^2}.$$

This is the sample standard deviation across the nine sectors' monthly returns on the same date. Higher dispersion indicates larger differences between sector winners and losers, giving context for relative-allocation decisions. It measures differences across sectors, whereas each ETF's volatility measures variation through time.

- **Average pairwise 63-day sector correlation (`sector_average_correlation_63d`).**

$$C_t = \frac{2}{N(N-1)}\sum_{i<j} \mathrm{Corr}_{63}(r_i^{(1)}, r_j^{(1)})_t.$$

The notebook computes Pearson correlations using the latest 63 daily simple returns and averages the 36 distinct sector pairs, excluding the diagonal. This describes how strongly sectors move together. Higher correlation suggests less diversification across sectors, while lower correlation indicates more independent movements. The quarterly window balances recent conditions with a longer sample for estimating co-movement.

- **VIX level (`vix_level`).**

$$V_t = \text{Close-}_{\texttt{VIX},t}\,.$$

This takes the downloaded VIX closing index level directly, without converting it into an asset return. It gives the policy an additional market-volatility signal alongside volatility estimated from past ETF returns. It is used as context, and the policy does not trade the VIX.

- **5-day VIX return (`vix_return_5d`).**

$$v_t^{(5)} = \frac{V_t}{V_{t-5}} - 1.$$

This is the proportional change in the VIX index over one trading week. It distinguishes a rising volatility environment from a falling one, even when their current index levels are similar. Its horizon matches the policy's decision interval. It is an index change, rather than the return from an investable VIX position.

Portfolio accounting and observation preparation.

- **One-day simple return (`return_1d`).**

$$r_{i,t}^{(1)} = \frac{P_{i,t}}{P_{i,t-1}} - 1.$$

This is stored to calculate daily portfolio returns and weight drift, and it underlies beta and correlation estimates. It is not included as a separate policy feature in the current configuration. The observation at date t determines holdings for later returns, beginning with $t+1$.

- **Trailing standardization and clipping.** For each market-feature column x_t, set $W = 1{,}260$ and compute

$$\begin{aligned}
\mu_t &= \frac{1}{W}\sum_{k=1}^{W} x_{t-k}, \\
s_t &= \sqrt{\frac{1}{W}\sum_{k=1}^{W}(x_{t-k}-\mu_t)^2}, \\
z_t &= \min\!\left(10, \max\!\left(-10, \frac{x_t-\mu_t}{s_t}\right)\right).
\end{aligned}$$

The scaling code uses a population standard deviation (`ddof=0`), unlike the sample standard deviations used in the volatility and dispersion features. Scaling puts inputs with different units, such as VIX levels and decimal returns, on comparable numerical scales. The five-year window provides a substantial historical reference that adapts over time. Statistics exclude the current observation and all future observations, and the same trailing procedure is used during training and evaluation. Clipping bounds extreme standardized values to support numerical stability; it does not remove their observations.

- **Current portfolio weights.** The nine current sector weights $w_{i,t-}$ are appended directly to the scaled market features without trailing standardization. They show the policy its existing allocation, including any negative positions, so residual actions can depend on current holdings and the cost of changing them.

3.3 Chronological partitions

The most recent two years are reserved as a final holdout. Earlier observations are split chronologically, with 80% assigned to training before the scaling warm-up is removed. Consequently, the usable training rows are fewer than 80% of the final, standardized development dataset.

Partition	First date	Last date	Dates
Training	2004-04-08	2019-08-23	3,871
Development validation	2019-08-26	2024-09-30	1,283
Reserved final holdout	2024-10-01	2026-09-30	501

Table 3: Partitions recorded in the saved model split files.

Validation starts with an observation on August 26, 2019, so the saved return series contains 1,282 realized daily returns from August 27, 2019 through September 30, 2024. The tables in this report use that common return period.

4 Training

4.1 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is a reinforcement learning algorithm that learns a decision rule, called a *policy*, through interaction with an environment. In this project, the environment is the historical portfolio simulator: the agent observes market features and current holdings, proposes allocation changes, and receives a reward based on the resulting portfolio growth after costs. Training seeks to improve expected discounted cumulative reward across successive decisions.

PPO uses an *actor–critic* approach:

- **Actor (policy).** The policy $\pi_\theta(a \mid s)$ defines a distribution over actions given observation s, with trainable parameters θ. Sampling actions during training provides exploration. Here, the sampled actions are converted into changes in the nine sector weights.
- **Critic (value function).** The value function $V_\phi(s)$ estimates the expected discounted future reward from observation s. It supplies a baseline for assessing actions, and is trained against estimated return targets.
- **Advantage estimate.** The estimate $\widehat{A}_k$ measures how favorable an action's outcome was relative to the critic's baseline, using subsequent rewards and value estimates. Positive advantages encourage the actor to make those actions more likely; negative advantages encourage the opposite. Here, k indexes policy decisions, each normally spanning five trading days.

PPO collects a batch of experience with the current policy, then performs multiple minibatch optimization passes before collecting a new batch. Its clipped policy objective discourages excessive

changes to action likelihoods during those updates. Define the likelihood ratio

$$q_k(\theta) = \frac{\pi_\theta(a_k \mid s_k)}{\pi_{\theta_{\text{old}}}(a_k \mid s_k)}.$$

For continuous actions, the numerator and denominator are probability densities. The actor maximizes

$$L^{\text{clip}}(\theta) = \widehat{\mathbb{E}}_k \left[\min\Big(q_k(\theta)\widehat{A}_k,\ \text{clip}(q_k(\theta), 1-\epsilon, 1+\epsilon)\widehat{A}_k \Big) \right],$$

where ϵ controls clipping and $\widehat{\mathbb{E}}_k$ denotes the batch average. The minimum removes the incentive to push an already favorable likelihood change beyond the clipping threshold. This encourages conservative updates but does not impose a hard bound on total policy change.[1]

PPO is an on-policy method: new experience is generated with the latest policy after each update cycle. The Stable-Baselines3 implementation also normalizes advantages during training.[2] We use it as a practical starting point because it supports continuous actions and integrates with our Gymnasium simulator.

4.2 Learning setup

All models use PPO with a feedforward multilayer-perceptron policy (`MlpPolicy`). The policy outputs nine continuous actions in $[-1, 1]$. The model-specific action function converts these actions into changes in portfolio weights. Episodes begin with an equal-weight portfolio and sample random starting points within the training partition. Each episode spans up to five trading years (1,260 daily intervals), while the policy acts once every five trading days.

Setting	Current value
Requested training budget	300,000 environment steps per run
Learning rate	3×10^{-4}
PPO rollout length	2,048 environment steps
Minibatch size	64
Episode length	Five trading years, with random training starts
Decision interval	Five trading days; daily valuation and accounting
Initial main-model seed	42
Repeated-seed experiments	42, 7, 21, 123, and 2026
Evaluation policy	Deterministic actions over development validation

Table 4: Shared training settings. An environment step normally spans five trading days; it is not a single daily return. PPO can round the requested training budget up to finish a rollout.

4.3 Portfolio accounting and reward

Let w_t be the weights established for a daily return interval, w_{t^-} the weights before trading, and r_{t+1} the vector of next-day asset returns. Turnover is the full sum of absolute weight changes, counting both purchases and sales:

$$\tau_t = \|w_t - w_{t^-}\|_1.$$

[1] Schulman et al. (2017), *Proximal Policy Optimization Algorithms*, https://arxiv.org/abs/1707.06347.

[2] Stable-Baselines3 2.3.2, PPO documentation: https://stable-baselines3.readthedocs.io/en/v2.3.2/modules/ppo.html.

The environment charges a transaction-cost rate $c = 0.0005$ (5 basis points per dollar traded), plus a daily borrowing charge based on an annual rate $b = 0.01$ applied to short exposure. Net return and the daily reward are

$$r_{t+1}^{\text{net}} = w_t^\top r_{t+1} - c\tau_t - \frac{b}{252}\sum_i \max(-w_{i,t}, 0),$$
$$R_{t+1} = 100 \log\Big(1 + r_{t+1}^{\text{net}}\Big).$$

The PPO step reward sums these daily rewards across the decision interval. The objective therefore rewards compounded growth after costs, with no separate drawdown or volatility penalty. Between decisions, holdings drift with asset returns rather than being reset to constant weights every day.

Performance reporting includes compounded net return, annualized net return, annualized volatility, Sharpe ratio, maximum drawdown, turnover, exposure, and implementation costs. Annualization uses 252 trading days. The Sharpe ratio uses a zero risk-free rate. Reported annualized implementation cost is the mean daily transaction-plus-borrow charge multiplied by 252, rather than a separately compounded estimate of lost wealth.

5 Model 1: Long Only

The first main model learns changes to a fully invested long-only portfolio. For action vector a_t, the proposed weights are

$$\widetilde{w}_t = w_{t^-} + a_t - \overline{a}_t \mathbf{1}.$$

Subtracting the mean action makes the proposed change sum to zero. The proposal is then projected onto the probability simplex, enforcing nonnegative weights that sum to one. The model can concentrate in selected sectors, but it cannot short, borrow to increase exposure, or allocate to a cash asset.

The saved seed-42 run in `models/long_only/` achieved a 23.51% annualized net return, a 0.975 Sharpe ratio, and a maximum drawdown of 35.07% in magnitude. Its annualized turnover was 22.54 times portfolio value, with an annualized implementation cost of 1.13%. This establishes a functioning learned allocation baseline, while showing that unconstrained action sizes can generate substantial trading even without short positions.

6 Model 2: Long Short

The second model retains the same centered, unscaled residual action but permits negative weights. Net exposure remains approximately 100%. If a proposed portfolio exceeds 160% gross exposure, its positive positions are rescaled to 130% long exposure and its negative positions to 30% short exposure. When the proposal is already within the gross limit, it is retained. Thus the model can use less than the full short allowance.

The seed-42 run in `models/long_short/` produced a 19.37% annualized net return and a Sharpe ratio of 0.932, with a 33.10% maximum drawdown in magnitude. Annualized turnover increased to 45.74 times portfolio value, and implementation cost rose to 2.59% per year. Shorting expanded the available portfolios, but the large residual actions made trading expensive, and this run did not improve on the long-only run's return or Sharpe ratio.

Five saved unscaled runs provide a broader check. Across seeds 42, 7, 21, 123, and 2026, mean annualized net return was 17.07%, mean Sharpe was 0.822, and mean annualized turnover was 44.37. Their mean annualized implementation cost was 2.52%. These figures are recorded in the validation summary under `models/long_short_unscaled_actions_multiseed/`.

The exposure constraint is applied when actions establish new target weights. Daily drift can subsequently push gross exposure above 160%; the original long-short run records a maximum of approximately 164.71%. The current simulator therefore does not enforce a continuous daily gross-exposure ceiling.

7 Model 3: Long Short with Scaled Actions

7.1 Smaller changes to current holdings

The third model introduces an action scale α:

$$\widetilde{w}_t = w_{t^-} + \alpha \left(a_t - \overline{a}_t \mathbf{1} \right).$$

It uses the same long-short exposure mapping as Model 2. Scaling changes how far the policy moves from its current holdings at each decision; it does not impose a fixed final position size or a hard turnover cap.

The initial scaled run used $\alpha = 0.02$. For seed 42, annualized turnover fell from 45.74 to 4.79, a reduction of approximately 89.5% relative to the unscaled long-short run. Annualized implementation cost fell from 2.59% to 0.50%. Annualized net return increased to 20.91%, Sharpe increased to 0.989, and maximum drawdown improved to 30.24% in magnitude.

Metric	Long only	Long short	Scaled 0.02	Scaled 0.10
Cumulative net return	192.74%	146.12%	162.77%	174.67%
Annualized net return	23.51%	19.37%	20.91%	21.97%
Annualized volatility	24.83%	21.47%	21.56%	24.41%
Sharpe ratio	0.975	0.932	0.989	0.936
Maximum drawdown	−35.07%	−33.10%	−30.24%	−34.74%
Annualized turnover	22.54	45.74	4.79	16.69
Annualized impl. cost	1.13%	2.59%	0.50%	1.13%

Table 5: Saved seed-42 development-validation results. Turnover is expressed as multiples of portfolio value per year. Sources: the validation metrics under `models/long_only/`, `models/long_short/`, `models/long_short_scaled_actions/`, and `models/long_short_scaled/seed_42/`.

The five-seed results at $\alpha = 0.02$ show why a single run is insufficient: annualized returns range from 11.06% to 20.91%. Mean annualized return is 17.19%, close to the unscaled model's 17.07%, while mean turnover falls to 4.73 and mean implementation cost to 0.50%. The strongest repeated-seed finding at this scale is the reduction in trading; the mean return improvement is small.

7.2 Completed action-scale search

Notebook 05 trains a fresh model for each scale and seed. The saved initial grid contains four scales and 20 checkpoints; the extended grid adds three scales and 15 checkpoints. Each of the seven scales

has been evaluated with the same five seeds. Scales are ranked by mean annualized net log growth, $252 \text{ mean}[\log(1 + r^{\text{net}})]$, across seeds.

Action scale	Mean log growth	Mean annual return	Mean Sharpe	Mean turnover
0.01	0.1208	12.85%	0.655	2.46
0.02	0.1582	17.19%	0.834	4.73
0.05	0.1756	19.19%	0.909	10.27
0.10	**0.1866**	**20.52%**	**0.960**	**17.37**
0.15	0.1741	19.05%	0.904	21.67
0.20	0.1457	15.71%	0.759	26.92
0.30	0.1381	14.86%	0.724	31.61

Table 6: Five-seed means from `grid_summary.csv` in `models/long_short_action_scale_grid/` and `models/long_short_action_scale_grid_extended/`. Log growth is in natural-log units per year.

Of the tested scales, 0.10 gives the highest mean validation log growth. Smaller scales reduce trading further but also restrict portfolio adaptation; larger scales increase turnover and have lower mean growth in these experiments. This pattern supports an intermediate action scale for further development, although validation-based tuning does not establish its performance on new data. The active configuration currently uses $\alpha = 0.10$ and seed 42.

7.3 Benchmarks and current interpretation

The saved benchmark comparison evaluates the active seed-42 scaled model against an equal-weight portfolio of the nine ETFs, rebalanced every five trading days with the same transaction-cost rate, and SPY buy-and-hold with no ongoing trading cost. The model returned 21.97% annually with a Sharpe ratio of 0.936. Equal-weight sectors returned 14.92% with a Sharpe ratio of 0.785, and SPY returned 16.37% with a Sharpe ratio of 0.835. These figures are saved in `results/daily_validation_strategy_metrics.csv`.

The active run exceeds both simple benchmarks on return and Sharpe over this period, but its 34.74% maximum drawdown is larger in magnitude than SPY's 33.72%. Its average gross exposure is approximately 158.36%, so its performance also reflects leverage and cannot be attributed solely to better sector selection. The seed-42 long-only run still has the highest annualized return among the individual runs in the main comparison table. The scaled model's strongest current contribution is a controllable trade-off between adaptation and trading cost, supported by repeated experiments.

8 Future Additions

8.1 Final evaluation and robustness

The next evaluation should freeze the selected model settings and selection rule before opening the reserved October 2024–September 2026 holdout. A final report should distinguish that evaluation from the validation results used to choose the action scale. Additional priorities are walk-forward evaluation, repeated long-only runs, and comparisons across market regimes. Reporting uncertainty across seeds and testing feature groups separately would help identify which results are stable and

which signals contribute to allocation decisions.

Benchmarking can be expanded with passive long-short or exposure-matched portfolios, allowing the effect of leverage to be assessed separately from the allocation policy. Cost sensitivity, turnover, and concentration should remain part of the comparison alongside returns and drawdowns.

8.2 Portfolio realism and controls

The current simulator uses a flat trading-cost rate and uniform short-borrow rate. Future versions can add bid–ask spreads, slippage, market impact, security-specific borrow availability and fees, and an explicit financing and cash-account treatment. The current accounting drifts weights using gross asset returns while deducting costs from portfolio wealth; a more complete balance sheet should reconcile positions, cash, and net equity.

Other useful additions include per-sector concentration limits, configurable turnover limits, and daily enforcement of exposure limits. An explicit execution delay would also allow testing whether signals remain useful when trades occur after the close used to construct the observation. Any changes to the growth reward, such as risk or drawdown penalties, should be treated as new experiments and compared with the current objective.

8.3 Individual-stock implementation

A preliminary extension already exists in Notebook 06. It freezes the scaled seed-42 policy at $\alpha = 0.10$, preserves the ETF-derived market features, and implements its sector targets through equally weighted stock baskets. The policy observes the stock fund's actual sector exposures, allowing it to respond to the holdings' drift. Saved artifacts contain five stocks per sector, or 45 stocks in total.

This prototype selects companies using current Yahoo market capitalizations and sector classifications, while requiring complete validation-period price histories. Consequently, its historical results include look-ahead and survivorship bias. A stronger extension requires point-in-time sector membership, historical market capitalizations, delisted securities, and stock-specific execution and borrowing assumptions before drawing performance conclusions.

There is also a reproducibility issue to resolve: the current Notebook 06 code sets `TOP_N=2`, while its narrative and saved selection describe five stocks per sector. The selection setting, narrative, and regenerated outputs should be aligned before the next stock-fund comparison.

8.4 Experiment organization

The training and grid-search notebooks currently repeat substantial data and environment code. Moving that shared logic into reusable modules would make future changes easier to validate consistently. A common experiment manifest should connect each checkpoint to its data snapshot, full configuration, seed, and results. Focused checks for feature timing, action constraints, weight drift, and cost accounting would further support reproducible comparisons as the project expands.